Mentimeter

Laboratorul 10

Design Patterns - Command și Builder

Mentimeter

Instructions

Mentimeter

Command

- încapsulează un apel cu tot cu parametri într-o clasă cu interfață generică
- behavioral pattern - modifică felul în care se efectuează apelurile

Mentimeter

De ce este folosit acest pattern?

- pentru a ușura crearea de structuri de delegare, de callback, de apelare întârziată
- pentru a reține lista de comenzi efectuate
- accounting
- liste de Undo / Rollback pentru operații reversibile

Mentimeter

Exemple de utilizare

- sisteme de accounting/logging pentru tranzactii
- sisteme de undo
- mecanism ordonat de delegare, apel intarziat, callback

Mentimeter

Funcționare și necesitate

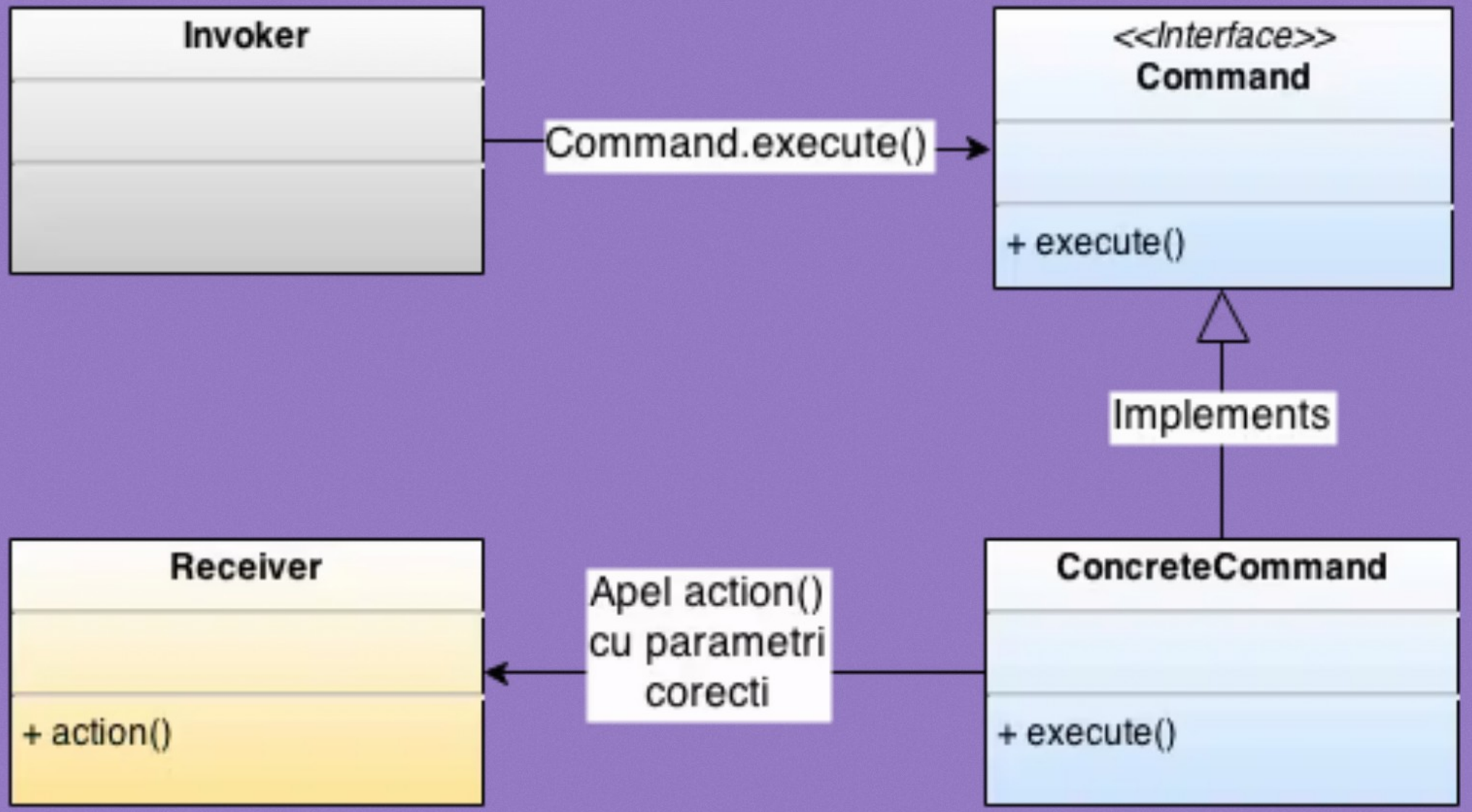
- încapsularea informațiilor referitoare la acțiuni/comenzi
- folosește un wrapper pentru a reține informația și a o folosi ulterior
- când vorbim de Command Pattern vorbim în general și de un Invoker

Mentimeter

Invoker

- managementul comenzilor
- listă de comenzi de ale căror dependințe va ține cont
- clientul va instanția comenzile și le va pasa invokerului (middleware între client și receiver)

Mentimeter



Structura Command Pattern

Mentimeter

Invoker

- comandantul
- apelează comenzile
- poate reține o listă de comenzi (pt undo/redo)
- primește clase Command pe care să le invoce

Mentimeter

Receiver

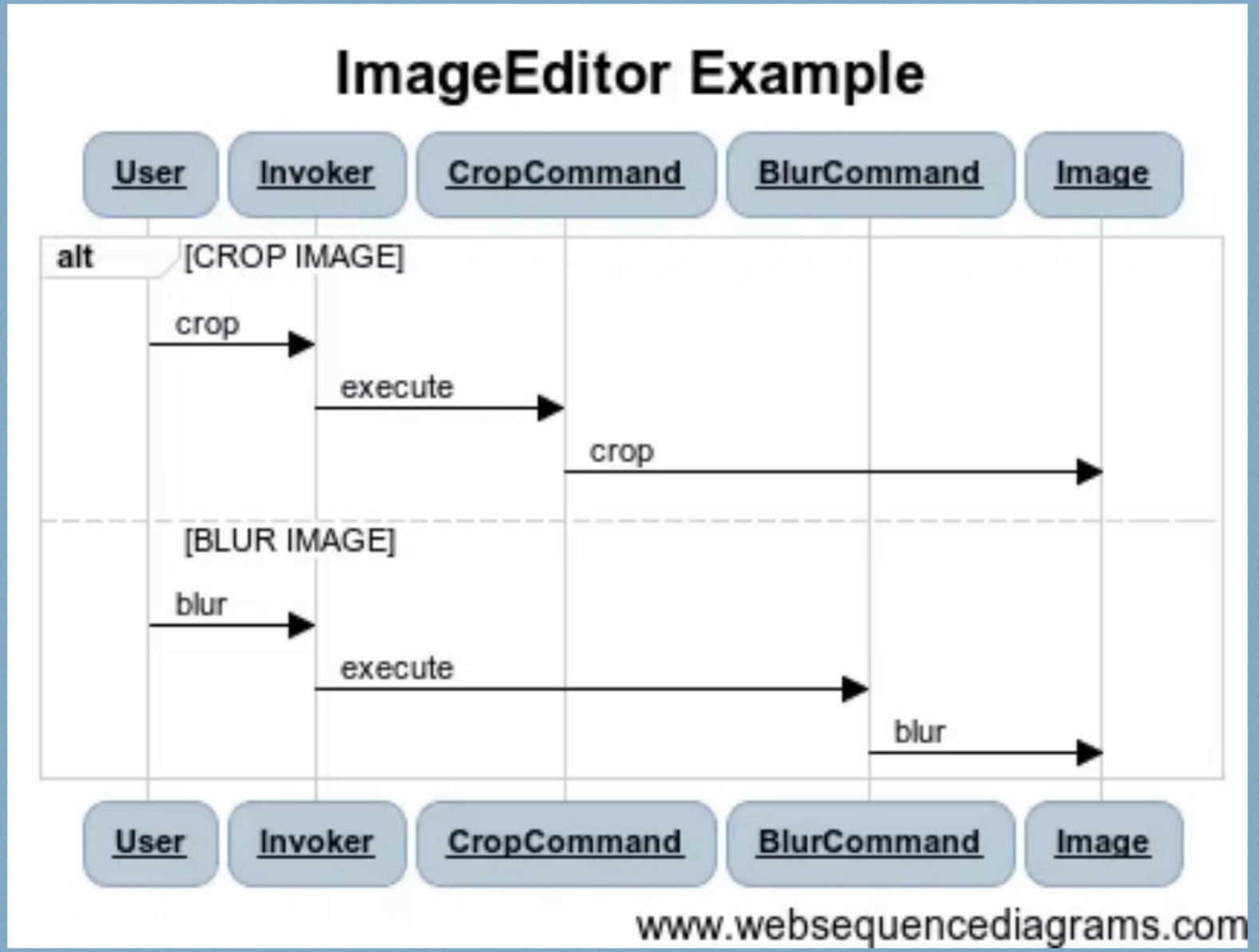
- comandatul
- clasa asupra căreia se face apelul
- conține implementarea efectivă a ceea ce se dorește executat

Mentimeter

Command

- conține metode pentru executarea comenzii (ex: execute())
- adăugăm metode pentru undo/redo pentru acțiunile undoable
- țin referințe către receivers pentru a invoca acțiunea

Mentimeter



Exemplu

Mentimeter

Demo

Mentimeter

Builder

- creational pattern
- folosit la crearea obiectelor complexe
- util pentru a separa proprietățile obligatorii de cele opționale

Mentimeter

```java
public class Pizza {
    private String pizzaSize;
    private int cheeseCount;
    private int pepperoniCount;
    private int hamCount;

    public Pizza(String pizzaSize, int cheeseCount, int pepperoniCount, int hamCount) {
        this.pizzaSize = pizzaSize;
        this.cheeseCount = cheeseCount;
        this.pepperoniCount = pepperoniCount;
        this.hamCount = hamCount;
    }

    public String getPizzaSize() {
        return pizzaSize;
    }

    public void setPizzaSize(String pizzaSize) {
        this.pizzaSize = pizzaSize;
    }

    public int getCheeseCount() {
        return cheeseCount;
    }

    public void setCheeseCount(int cheeseCount) {
        this.cheeseCount = cheeseCount;
    }
```

Exemplu

Mentimeter

```java
public class Pizza {
    private String pizzaSize;
    private int cheeseCount;
    private int pepperoniCount;
    private int hamCount;

    public Pizza(String pizzaSize) {
        this(pizzaSize,  cheeseCount: 0,  pepperoniCount: 0,  hamCount: 0);
    }

    public Pizza(String pizzaSize, int cheeseCount) {
        this(pizzaSize, cheeseCount,  pepperoniCount: 0,  hamCount: 0);
    }

    public Pizza(String pizzaSize, int cheeseCount, int pepperoniCount) {
        this(pizzaSize, cheeseCount, pepperoniCount,  hamCount: 0);
    }

    // ... (constructor pentru orice combinatie de parametrii)

    public Pizza(String pizzaSize, int cheeseCount, int pepperoniCount, int hamCount) {
        this.pizzaSize = pizzaSize;
        this.cheeseCount = cheeseCount;
        this.pepperoniCount = pepperoniCount;
        this.hamCount = hamCount;
    }
```

Supraîncărcarea constructorilor

Mentimeter

```java
public class Main {

    public static void main(String[] args) {
        Pizza pizza = new Pizza();
        pizza.setCheeseCount(1);
        pizza.setPizzaSize("extra");
        pizza.setHamCount(1);
    }
}
```

Folosirea de getters și setters

Mentimeter

```java
public class Pizza {
    private String pizzaSize;
    private String crust;
    private int cheeseCount;
    private int pepperoniCount;
    private int hamCount;
    private int mushroomsCount;

    public static class Builder {
        private String pizzaSize;
        private String crust;
        private int cheeseCount = 0;
        private int pepperoniCount = 0;
        private int hamCount = 0;
        private int mushroomsCount = 0;

        public Builder(String pizzaSize, String crust) {
            this.pizzaSize = pizzaSize;
            this.crust = crust;
        }

        public Builder cheeseCount(int cheeseCount) {
            this.cheeseCount = cheeseCount;
            return this;
        }
```

Builder 1

Mentimeter

```
        public Pizza build() {
            return new Pizza( builder: this);
        }
    }

    private Pizza(Builder builder) {
        this.pizzaSize = builder.pizzaSize;
        this.crust = builder.crust;
        this.cheeseCount = builder.cheeseCount;
        this.pepperoniCount = builder.pepperoniCount;
        this.hamCount = builder.hamCount;
        this.mushroomsCount = builder.mushroomsCount;
    }
}
```

Builder 2



Mentimeter

```
public class Main {

    public static void main(String[] args) {
        Pizza pizza = new Pizza.Builder( pizzaSize: "large",  crust: "thin")
                .cheeseCount(1)
                .pepperoniCount(1)
                .build();
    }
}
```

Builder 3